Data Structures and Algorithms 2

Question 3: Recursion

GitHub repository: https://github.com/janadatatech/dsa2-coursework

Coursework Question 3: Recursion (20 marks)

You must create a function that follows the following logic:

The function receives a list of strings. Each string can be either “blue”, “red” or “green”. Your goal is to move through the list incrementing the index by following the instructions below:

- If the string at index i is “blue”, the next element is the element following the next appearance of the string “blue”.
- If the string at index i is “red”, the next element is the element at index 2 * i + 1.
- If the string at index i is “green”, the next element is the following element in the list, i.e. at index i + 1.

The output should be a list with all the indexes of the elements you visited, in order.

Example

For the list ["blue", "green", "red", "blue", "green", "red"], the sequence of steps would be:

1. The first step is the first element, “blue”, at index 0. Search for the next “blue” and take the following element. The next “blue” is at index 3, followed by “green” at index 4.
2. The second element is “green” at index 4. Go to the next element, which is “red” at index 5.
3. The third element is “red” at index 5. Go to index 2 * 5 + 1, which is 11. This index does not exist, so the program exits.

The output is [0, 4, 5].

Tasks

a) Code the function using recursion. Answers without using recursion will score 0 for this question. Please use the following signature:

```
def follow_instructions(instructions: list[str]) -> list[int]
```

i. Test it with 2 generated lists.

b) Create the flowchart of your code and put it in the report.

Q3(a): Recursive approach

I start at index 0 and use a list to store the visited indexes and the If statements check the colour:

If Green: move to i + 1, If Red: move to 2 * i + 1 If Blue: find the next blue and move to the position after it.

The "visit" function calls itself with the next index which is the recursion and the function stops when the index reaches the end of the list or goes beyond it.

It would also stop if there is no next blue which is the assumption I'm working on for the question

```
def follow_instructions(instructions: list[str]) -> list[int]:
    visited = []

    def visit(index):
        if index >= len(instructions):  #stop when the index reaches the end
            return
        visited.append(index) # Record current index.
        colour = instructions[index]
        if colour == "green":
            visit(index + 1)
        elif colour == "red":
            visit(2 * index + 1)
        elif colour == "blue":
            for position in range(index + 1, len(instructions)): # Search for the next blue.
                if instructions[position] == "blue":
                    visit(position + 1)
                    return

    visit(0)
    return visited
```

Check example

I call the function using the example list from the question.

The expected result is [0, 4, 5] and I compare this with the returned list to check the result.

```
example = ["blue", "green", "red", "blue", "green", "red"]
result = follow_instructions(example)
print("Visited indexes:", result)
print("Matches expected result:", result == [0, 4, 5])
```

```
Visited indexes: [0, 4, 5]
Matches expected result: True
```

Test 1: Check all three colours

I test a list containing green, red and blue.

The expected result is [0, 1, 3, 6] and the function then reaches index 7 and stops because it is outside the list.

I use == to check whether the result matches the expected list.

```
test1 = ["green", "red", "green", "blue", "red", "blue", "green"]
result1 = follow_instructions(test1)
expected1 = [0, 1, 3, 6]
print("Result:", result1)
print("Expected:", expected1)
print("Test passed:", result1 == expected1)
```

```
Result: [0, 1, 3, 6]
Expected: [0, 1, 3, 6]
Test passed: True
```

Test 2: No next blue

The function visits indexes 0 and 1, then stops because there is no next blue for which I use == to compare the result with the expected list [0, 1].

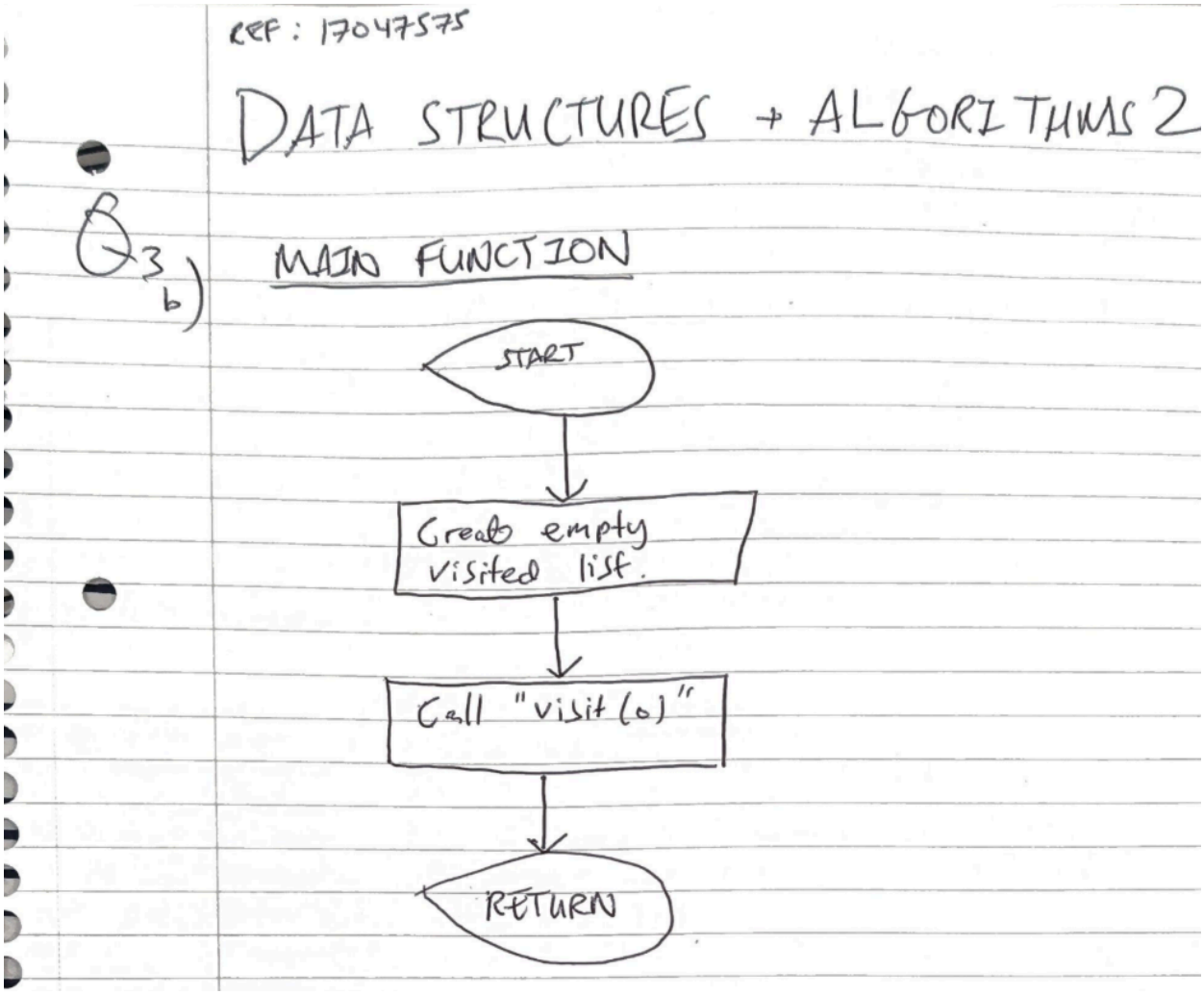
```
test2 = ["green", "blue", "red", "green", "red"]
result2 = follow_instructions(test2)
expected2 = [0, 1]
print("Result:", result2)
print("Expected:", expected2)
print("Test passed:", result2 == expected2)
```

```
Result: [0, 1]
Expected: [0, 1]
Test passed: True
```

Q3(b): Flowcharts

Main function

The main function creates the visited list and it calls "visit(0)" to start at index 0, then returns the list.



Recursive function

The function adds the current index to the visited list and If statements check the colour.

A for loop searches for the next blue and the function calls itself with the next index.

It stops at the end of the list or when there is no next blue.

